

Vision-Language Geometric Programming for Long-Horizon Robotic Assembly

Michael Shell, *Member, IEEE*, John Doe, *Fellow, OSA*, and Jane Doe, *Life Fellow, IEEE*

Abstract—The abstract goes here.

Index Terms—IEEE, IEEEtran, journal, LATEX, paper, template.

I. INTRODUCTION

THIS demo file is intended to serve as a “starter file” for IEEE journal papers produced under LATEX using IEEEtran.cls version 1.8b and later.

II. RELATED WORK

III. PRELIMINARIES AND PROBLEM STATEMENT

While Logic-Geometric Programming (LGP) provides a mathematically rigorous framework for tightly coupling discrete logic with continuous trajectories, applying it off-the-shelf to long-horizon real-world assembly exposes two major computational bottlenecks:

1) Combinatorial Explosion of the Search Space: The state-space search complexity scales as $\mathcal{O}(|A|^D)$, where $|A|$ is the size of the available action library and D is the plan depth. In deep manual-guided assembly tasks, lacking a strong semantic heuristic forces the underlying solver into frequent, costly geometric back-tracking evaluating physically irrelevant predicates, easily pushing computation times from minutes to hours.

2) Singularities in Closed Kinematic Loops: Multi-support assembly tasks inherently induce closed kinematic loops (e.g., placing a long block across two pillars). Native LGP assumes strict tree topologies; forcing dynamic topology switches in such scenarios often violates the tree-based assumptions of the k -order Markov Optimization (KOMO) engine, leading to solver deadlocks.

3) Physical Brittleness of Pure Semantic Planning: While recent Vision-Language Models (VLMs) excel at generating high-level sub-goals, they lack strict geometric grounding and frequently hallucinate or omit essential physical preconditions (e.g., failing to output an “open drawer” action before picking an object inside). Thus, relying solely on feed-forward semantic plans inevitably leads to execution failures. A robust framework must treat the VLM’s output as a highly pruned “task skeleton” rather than an absolute command, relying on a deeply integrated geometric solver to autonomously fill in the missing physical gaps to guarantee kinematic feasibility.

M. Shell was with the Department of Electrical and Computer Engineering, Georgia Institute of Technology, Atlanta, GA, 30332 USA e-mail: (see <http://www.michaelshell.org/contact.html>).

J. Doe and J. Doe are with Anonymous University.

Manuscript received April 19, 2005; revised August 26, 2015.

IV. METHOD

We propose **V-LGP** (Vision-Language Geometric Programming), a hierarchical framework that bridges high-level semantic reasoning with low-level geometric motion planning.

A. Phase 0: Scene Initialization and Semantic Grounding

To bridge the continuous visual observation space with the discrete symbolic domain required by the LGP solver, the pipeline aligns top-down visual inputs with an uninstantiated kinematic tree (e.g., an anonymous .g configuration). Similar to recent VLM-guided TAMP frameworks [8] that establish scene properties prior to planning, we process object bounding boxes and ground them to semantic simulation IDs. Specifically, objects are spatially sorted via their radial Euclidean distance from the robot base. This autonomous visual-to-symbolic mapping provides a deterministic anchor for all subsequent geometric constraints without requiring predefined simulator states.

B. Phase 1: Extraction of the Layered Precedence Graph

Listing 1. Example of the Layered Precedence Graph extracted by the VLM Analyst. This structured schema restricts the geometric search depth

```
{
  "assembly_nodes": [
    {
      "node_id": 1,
      "description": "Foundation",
      "actions": [
        { "object": "base", "placed_on": "table", "at_slot": "place_base1" },
        { "object": "cylinder", "placed_on": "base", "at_slot": "top_left" }
      ]
    },
    {
      "node_id": 2,
      "description": "Second Layer",
      "actions": [
        { "object": "base", "placed_on": ["cylinder"], "at_slot": null },
        { "object": "cylinder", "placed_on": "base", "at_slot": "mid_left" }
      ]
    }
  ]
}
```

```

17 ]
18 }
19 ]
20 }

```

In complex multi-part assembly tasks, restricting the state-space search via sequence-defining precedence graphs is an established methodology (e.g., as explored in systems like Fabrica [10]). While existing approaches often rely on dedicated physics-based disassembly planners, we propose an alternative semantic framework. Instead of requiring intensive kinematic pre-computations, a Vision-Language Model acts as a high-level zero-shot parser to process visual assembly instructions.

Similar to the sub-goal temporal decomposition seen in hierarchical planning [4], the VLM extracts a semantic *Layered Precedence Graph* $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ (see Listing 1). Each node $v \in \mathcal{V}$ encapsulates a disjoint assembly stage (e.g., placing the foundation before the second layer), generating an ordered sequence of semantic placement tasks. This graph serves as an abstract temporal skeleton for downstream processing.

C. Phase 2: Dynamic Resource Allocation and Batch Compilation

To overcome the combinatorial explosion identified in Section III, we introduce a dynamic pruning pipeline comprising a *VLM Strategist* and a *VLM Decider*.

For each task node extracted in Phase 1, the **VLM Strategist** acts as a robotic logistics planner. It first performs *Resource Allocation*, actively binding generic object semantic requests (e.g., “cylinder”) to uniquely available physical entities in the live simulation inventory (e.g., `cyl1` or `cyl2`). Subsequently, it generates multiple valid sequential execution strategies evaluated by heuristics like spatial accessibility (e.g., favoring top-to-bottom manipulation to minimize arm occlusion).

The optimal sequence is then passed to the **VLM Decider**, which functions as a safety officer and logic compiler. The Decider splits the sequence into hardware-safe batches guided by strict dependency and structural hierarchy rules (e.g., segregating foundational bases from secondary components). Most importantly, the Decider explicitly outputs flat, atomic facts adhering to native LGP syntax without nested boolean logic (e.g., generating `(on top_left_base1 cyl1)` alongside designated action rules like `pick_cylinder`).

By proactively injecting these structured batches as localized goal bounds, V-LGP fundamentally bypasses the global state-space search inherent to standard LGP. The search tree depth is clamped to the scale of individual batches ($|P_{\text{batch}}| \ll |P|$), regularizing convergence predictability and dropping the original exponential execution time to deterministically scalable limits.

Crucially, this architecture distinctly differentiates V-LGP from standard VLM-guided TAMP frameworks. While the VLM utilizes physical common sense to drastically reduce combinatorial possibilities, the underlying LGP engine retains its unique capacity to tightly integrate discrete transitions with continuous motion. This unified representation absorbs the

inherent ambiguity of VLM instructions, offering sufficient redundancy to dynamically adjust continuous waypoints and resolve local geometric conflicts autonomously.

D. Phase 3: Constrained Geometric Execution

The geometric execution phase translates the logic bounds dynamically compiled in Phase 2 into continuous trajectory optimization using the native C++ LGP infrastructure [1]. Because the VLM has strictly pruned the logical permutations, the k -order Markov Optimization (KOMO) solver focuses solely on resolving local geometric feasibility. The core action skeleton is refined through three specialized, physics-grounded manipulation primitives.

1) *Semantic-Guided Auxiliary Grasping* (`action_pick`): To handle large-scale objects, standard LGP solvers often fail during inverse kinematics if targeting the ungraspable object’s geometric centroid. We introduce a decoupled *Semantic Grasping Site* paradigm. During `action_pick`, the system traverses the object’s kinematic tree and re-parents the optimization target to an auxiliary proxy frame, *targetF* (e.g., a designated handle). The grasp pose leverages a 6-DOF free joint relative to the gripper:

$$f_{\text{snap}} \leftarrow \text{initFrameDof}(\text{gripper}, \text{targetF}) \quad (1)$$

This transforms a highly non-convex full-body collision problem into a tractable sub-problem focused tightly on the valid grasping manifold.

2) *Topological Decoupling for Multi-Support* (`action_place_multi`): Native LGP assumes strict tree topologies, which fail in multi-support structural scenarios. We resolve this by introducing a highly adaptable unactuated 6-DOF *Virtual Anchor* (A_v) parented directly to the world frame.

For placing an object O onto a composite set of support frames $S_{1..n}$, A_v is assigned an initial pose matching the calculated target centroid. The solver then enforces a soft optimization constraint aligning A_v to the target world pose, while employing a rigid kinematic switch allocating ownership of object O to A_v :

$$\Phi_{\text{anchor}} = \|\phi_{\text{poseDiff}}(A_v, O, x_t)\|^2 \approx 0 \quad (2)$$

This manipulation elegantly breaks closed kinematic loops, granting the KOMO solver dynamic capability to iteratively interpolate the most stable geometric descent.

3) *Trajectory Shaping and Three-Stage Safety Protocol*: To overcome solver deadlocks caused by static long-horizon obstacles, all generated sequences are heavily regularized through dynamic trajectory shaping:

Pre-Action Funnels: To prevent lateral sliding collisions during delicate placements or insertions, we mathematically inject relative positional waypoints at $t - 0.5$ and $t - 0.1$. This constructs a virtual “funnel” forcing the payload precisely above the anchor with a strict vertical standoff distance (e.g., $d_z = 0.05$ m):

$$\Phi_{\text{funnel}} = \|\phi_{\text{posRel}}(O, A_v, x_{t-0.1}) - [0 \ 0 \ d_z]^T\|_{\text{sos}}^2 \quad (3)$$

Three-Stage Phased Collision Relaxations: Furthermore, continuous collision constraints $FS_{\text{negDistance}}$ between the

moving gripper parts and static obstacles are segmented temporally. During the *transit phase* ($t \in [t - 1.0, t - 0.2]$), strict safety margins ($d_{safe} \geq 0.03\text{m}$) are enforced utilizing bounding hierarchies. However, within the *final approach phase* ($t - 0.2$ to t), these margins are hierarchically relaxed to allow physical manifold contact, enabling deadlock-free grasping and precision placement.

E. Phase 4: State Verification and Surgical Correction

Given the inherent uncertainties of physical execution (e.g., slipping, grasping failures), open-loop trajectory execution is insufficient for long-horizon assembly. To address this, we introduce a *VLM Judicator* that performs state verification and surgical correction after each executed batch.

The Judicator captures a top-down visual observation of the current workspace and compares it against the expected blueprint state established in Phase 1. If a deviation is detected—such as a missing component in a designated slot or an unexpectedly placed object (an “intruder”)—the system halts the standard planning pipeline and initiates a targeted correction sequence.

Relying on 2D spatial reasoning, the Judicator maps the observed visual error back to the symbolic domain. It dynamically formulates a deterministic recovery action: either relocating the misplaced object to its correct slot (e.g., `(on target_slot obj)`) or safely removing an unrecognized entity to the neutral table area (e.g., `(on table obj)`). Finally, it compiles this corrective action into a singular, hardware-safe LGP batch containing exclusively the required `pick` and `place` decision rules. By embedding these structural checkpoints at the end of every placement layer, the framework prevents minor execution errors from cascading into catastrophic structural failures, offering a pragmatic mechanism for closed-loop error recovery.

V. EXPERIMENTAL RESULTS

A. Performance Evaluation

Experimental data and baseline comparisons with native LGP go here.

VI. CONCLUSION

The conclusion summarizes the contributions of V-LGP in long-horizon assembly tasks.

APPENDIX A

PROOF OF THE FIRST ZONKLAR EQUATION

Appendix one text goes here.

ACKNOWLEDGMENT

The authors would like to thank...

REFERENCES

- [1] M. Toussaint, “Logic-geometric programming for multi-step manipulation,” in *Proceedings of the International Joint Conference on Artificial Intelligence (IJCAI)*, 2015.
- [2] M. Toussaint, “Newton methods for k-order Markov optimization,” *arXiv preprint arXiv:1407.0414*, 2014.
- [3] J. Schoeler, F. T. Rehder, and M. Toussaint, “R-LGP: Reactive Logic-Geometric Programming,” *arXiv preprint arXiv:2310.02791*, 2023.
- [4] J. Doe et al., “Receding Horizon Hierarchical Logic-Geometric Programming,” *arXiv preprint arXiv:2110.03420*, 2021.
- [5] F. Lin et al., “Text2Motion: From Natural Language to Logic-Geometric Programming,” *arXiv preprint arXiv:2303.12153*, 2023.
- [6] General VLA Reference, “Vision-Language-Action Models for Robotics,” *Placeholder*, 2024.
- [7] H. Kopka and P. W. Daly, *A Guide to L^AT_EX*, 3rd ed., 1999.
- [8] M. Toussaint et al., “General VLA Reference for Task and Motion Planning,” *arXiv preprint arXiv:2410.02193*, 2024.
- [9] J. Doe et al., “Vision-Language Models for Robotic Assembly,” *arXiv preprint arXiv:2407.09829*, 2024.
- [10] Y. Zhu et al., “Fabrica: Dual-Arm Assembly of General Multi-Part Objects via Integrated Planning and Learning,” *arXiv preprint arXiv:2506.05168*, 2024.